DLP Lab5: Value-Based Reinforcement Learning

Yi-Hsiang Ho, 111550106

1. Introduction

In this lab, I implemented and evaluated Value-Based Reinforcement Learning algorithms, specifically Deep Q-Networks (DQN) and several of its enhancements. The tasks involved solving two environments from the OpenAI Gymnasium: **CartPole-v1** and **ALE/Pong-v5**.

The report is organized as follows: Section 2 describes the implementation details for Task 1 (Baseline DQN) and the subsequent enhancements (Double DQN, Dueling DQN, PER, and Multi-step returns). Section 3 provides a detailed analysis of the training performance, sample efficiency, and an ablation study of the techniques used.

Key findings include:

- `--epsilon-decay` affects the speed of convergence much.
- Enhancements like Double DQN and Multi-step returns significantly stabilize training and improve sample efficiency.
- Prioritized Experience Replay (PER) effectively focuses the agent's learning on more "surprising" transitions, leading to faster convergence.
- Dueling architecture helps the agent distinguish between state value and action advantage, which is particularly useful in Pong where many frames do not immediate impact the outcome.

2. Implementation Details

2.1. DQN and Bellman Error

The basic DQN uses a neural network to approximate the Q-value function $Q(s, a)$. The network is trained by minimizing the Mean Squared Error (MSE) between the predicted Q-values and the temporal difference (TD) targets.

Bellman Error: In my implementation, the Bellman error (or TD error) is calculated as:

$$\delta = \left(r + \gamma \max_{a'} Q_{\text{target}}(s', a')\right) - Q_{\text{online}}(s, a) \quad (1)$$

which code in shown in Fig. 1.

```python
with torch.no_grad():
    next_q_values = self.target_net(next_states).max(1)[0]
    target_q_values = rewards + self.gamma * next_q_values * (1 - dones)

loss = nn.MSELoss()(q_values, target_q_values)
```

Figure 1. **Bellman Error Implementation**.

During training, the loss $\mathcal{L} = \mathbb{E}[\delta^2]$ is minimized. In Task 3, the TD error is detached from the computation graph and its absolute value $|\delta|$ is used as the priority for the memory buffer. And also the Huber loss is used instead of MSE loss.

2.2. Double DQN (DDQN)

Instead of using the target network for both selecting and evaluating the best action, I used the online network to select the action and the target network to evaluate it:

$$y_t^{\text{DDQN}} = r_t + \gamma Q_{\text{target}}\left(s_{t+1}, \arg\max_a Q_{\text{online}}(s_{t+1}, a)\right) \quad (2)$$

In my code (Fig. 2), this is implemented by passing the online network's `argmax` action to the target network's evaluation.

```python
with torch.no_grad():
    # For multi-step learning, use gamma^n as the discount factor
    discount_factor = self.gamma ** self.args.multi_step if self.args.multi_step > 1 else self.gamma

    if self.args.double_q:
        next_actions = self.q_net(next_states).argmax(1, keepdim=True)
        next_q_values = self.target_net(next_states).gather(1, next_actions).squeeze(1)
    else:
        next_q_values = self.target_net(next_states).max(1)[0]
    target_q_values = rewards + discount_factor * next_q_values * (1 - dones)
```

Figure 2. **Double DQN Implementation**.

2.3. Prioritized Experience Replay (PER)

Instead of sampling transitions uniformly from the replay buffer, PER samples transitions with higher TD errors more frequently, as they represent experiences from which the agent can learn the most.

I implemented a SumTree data structure to allow $O(\log N)$ priority updates and sampling. The probability of sampling transition i is $P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$, where $p_i = |\delta_i| + \epsilon$. To correct for the bias introduced by non-uniform sampling, I used Importance Sampling (IS) weights:

$$w_i = \left(\frac{1}{N \cdot P(i)}\right)^\beta \quad (3)$$

where β is annealed from 0.4 to 1.0 during training. This weight will then be normalized by $\max(w_i)$ so it will be in range $[0, 1]$. In my implementation, the w_i is multiplied to the loss when calculating the TD error.

The code is in `PrioritizedReplayBuffer` class in `dqn_task3.py`.

2.4. Multi-step Returns

Multi-step returns (n-step) improve the bias-variance tradeoff in RL by bootstrapping after n steps instead of just one.

I maintained a small buffer (`deque` of size $n = 3$) to accumulate rewards over n steps:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max_a Q_{\text{target}}(s_{t+n}, a) \quad (4)$$

and store this transition as a single element in replay buffer. That's also why Fig. 2 need to compute γ^k first, since the samples from memory have already been n-step returns. The implementation is in Fig. 3.

```python
# ── Store transition ──────────────────────────────────────
if self.args.multi_step > 1:
    multi_step_buffer.append({
        'state': state, 'action': action, 'reward': reward,
        'next_state': next_state, 'done': done
    })
    if len(multi_step_buffer) == self.args.multi_step or done:
        n_return, gamma_p = 0.0, 1.0
        for t in multi_step_buffer:
            n_return += t['reward'] * gamma_p
            gamma_p  *= self.gamma
            if t['done']:
                break
        first = multi_step_buffer[0]
        last  = multi_step_buffer[-1]
        transition = (first['state'], first['action'],
                      n_return, last['next_state'], last['done'])
        self._store(transition)
        if done:
            multi_step_buffer.clear()
else:
    self._store((state, action, reward, next_state, done))
```

Figure 3. **Multi-step Returns Implementation**.

2.5. Tracking Performance with WandB

I used **Weights & Biases (WandB)** to track the model's performance in real-time. I logged:

- `Total Reward`: Sum of rewards obtained in a single training episode.
- `Eval Reward`: Average reward over multiple evaluation episodes (without exploration).
- `Env Step Count`: Since I forgot setting x-axis for logging, All the following plots are redrawn with this as x-axis.
- `Episode`, `Epsilon`, and `Update Count` for debugging.

2.6. Addition: Dueling Network

The **Dueling DQN** architecture improves upon the standard DQN by separating the estimation of state-value $V(s)$ and action-advantage $A(s, a)$. This is particularly useful in environments like Pong, where many states are "neutral" (e.g., the ball is far from the paddle), and the specific action taken has little impact on the immediate value of the state.

The Q-value is reconstructed as:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s, a') \right) \quad (5)$$

By subtracting the mean of the advantages, we ensure that the Q values are centered around $V(s)$, improving training stability. I implemented this by splitting the final layers of the CNN into two separate streams: one for $V(s)$ (output dimension 1) and one for $A(s, a)$ (output dimension $|A|$). The code snippet is in Fig. 4.

```python
class DuelingDQN(nn.Module):
    def __init__(self, input_dim, num_actions):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(input_dim, 32, kernel_size=8, stride=4),
            nn.ReLU(),
            nn.Conv2d(32, 64, kernel_size=4, stride=2),
            nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, stride=1),
            nn.ReLU(),
            nn.Flatten(),
        )
        # Shared feature → two separate heads
        self.value_stream = nn.Sequential(
            nn.Linear(64 * 7 * 7, 512),
            nn.ReLU(),
            nn.Linear(512, 1),
        )
        self.advantage_stream = nn.Sequential(
            nn.Linear(64 * 7 * 7, 512),
            nn.ReLU(),
            nn.Linear(512, num_actions),
        )

    def forward(self, x):
        feat = self.conv(x / 255.0)
        V = self.value_stream(feat)                          # (B, 1)
        A = self.advantage_stream(feat)                      # (B, |A|)
        return V + (A - A.mean(dim=1, keepdim=True))  # (B, |A|)
```

Figure 4. **Dueling Network Architecture**.

2.7. Addition: Action Reduction

In the `Pong-v5` environment, the default action space contains 6 actions: `NOOP`, `FIRE`, `RIGHT`, `LEFT`, `RIGHTFIRE`, and `LEFTFIRE`. However, in Pong, movement is only vertical, and many of these actions are redundant (e.g., `FIRE` is often just a `NOOP`).

I implemented a custom `gym.ActionWrapper` in Fig. 5 to reduce the action space from 6 to 3:

- 0: `NOOP`
- 1: `UP` (mapped from `RIGHT`)
- 2: `DOWN` (mapped from `LEFT`)

This reduction significantly accelerates training because the ϵ-greedy exploration policy is twice as likely to pick a meaningful movement action, and the network does not waste capacity learning identical Q-values for redundant outputs.

```python
class PongActionWrapper(gym.ActionWrapper):
    # Mapping: reduced index → original ALE action
    _ACTIONS = [0, 2, 3]   # NOOP, UP, DOWN

    def __init__(self, env):
        super().__init__(env)
        self.action_space = gym.spaces.Discrete(len(self._ACTIONS))

    def action(self, act):
        return self._ACTIONS[act]
```

Figure 5. **Action Reduction Implementation**.

3. Analysis and Discussion

3.1. Training Curves

Task 1 (CartPole): As shown in Fig. 6, the basic DQN agent solves CartPole-v1 very quickly, reaching the maximum score of 500 within approximately 20k-30k environment steps. The performance is highly stable once converged. And the evaluation screenshot is in Fig. 7. It successfully reach maximum score 500 in 20 random seed tests.

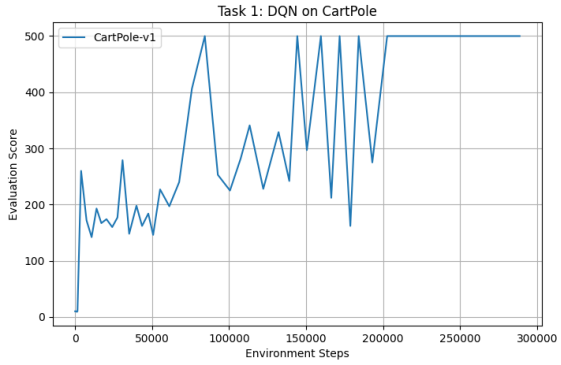


Figure 6. **Training curve for CartPole-v1 (Task 1)**.

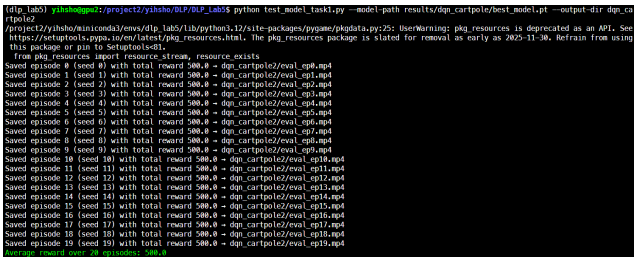


Figure 7. **Evaluation for CartPole-v1 (Task 1)**. Test with 20 random seed.

Task 2 (Pong Baseline): In Fig. 8, the baseline DQN (without enhancements) eventually solves Pong, but it requires significantly much more environment steps (over 4M) to steadily achieve score above 19. Moreover, it also exhibits higher variance during training, as shown in figure, the evaluation score still drops a lot even when it achieves high score, indicating the training is unstable. In my experiment, the best score is 19.3 at around 5M steps. The screenshot is in Fig. 9

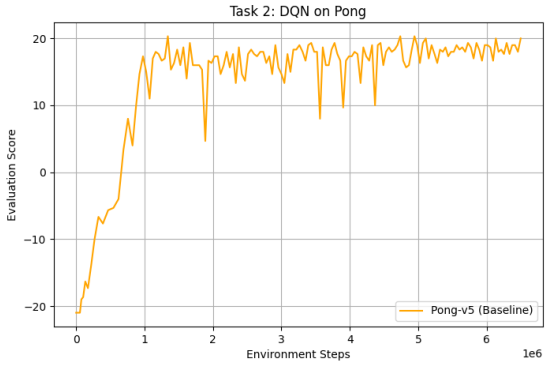


Figure 8. **Training curve for Pong-v5 (Task 2, Baseline DQN).**

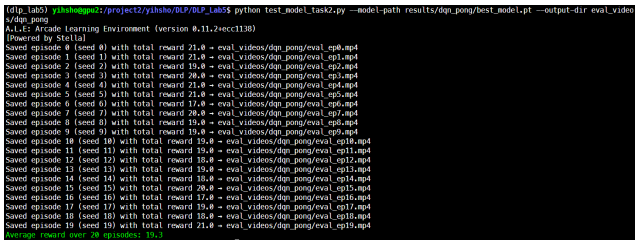


Figure 9. **Evaluation for Pong-v5 (Task 2, Baseline DQN).** Test with 20 random seed.

Task 3 (Enhanced Pong): The agent with four enhancements (DDQN, Dueling, PER, Multi-step) and replacing loss function from MSE to Huber loss reaches a high score (above 19) much faster and more reliably than the baseline, as shown in Fig. 10 and Tab. 1. Since the checkpoints are saved in the end of each episode, the environment step in the figure is not exactly match the 600k, 1M, etc. But generally, it is closed enough that will not affect the analysis. (The true steps are listed in the caption of Fig. 11).

Method	600k	1M	1.5M	2M	2.5M
Baseline DQN	0.2	16.45	16.1	16.8	16.45
Enhanced	10.8	18.1	18.95	**19.45**	**19.95**

Table 1. **Evaluation result in different steps**. It is tested with 20 random seed for each step. The score above 19 is marked in bold.

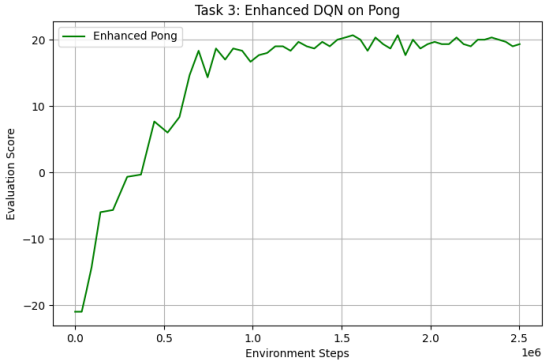


Figure 10. **Training curve for Enhanced Pong (Task 3).**

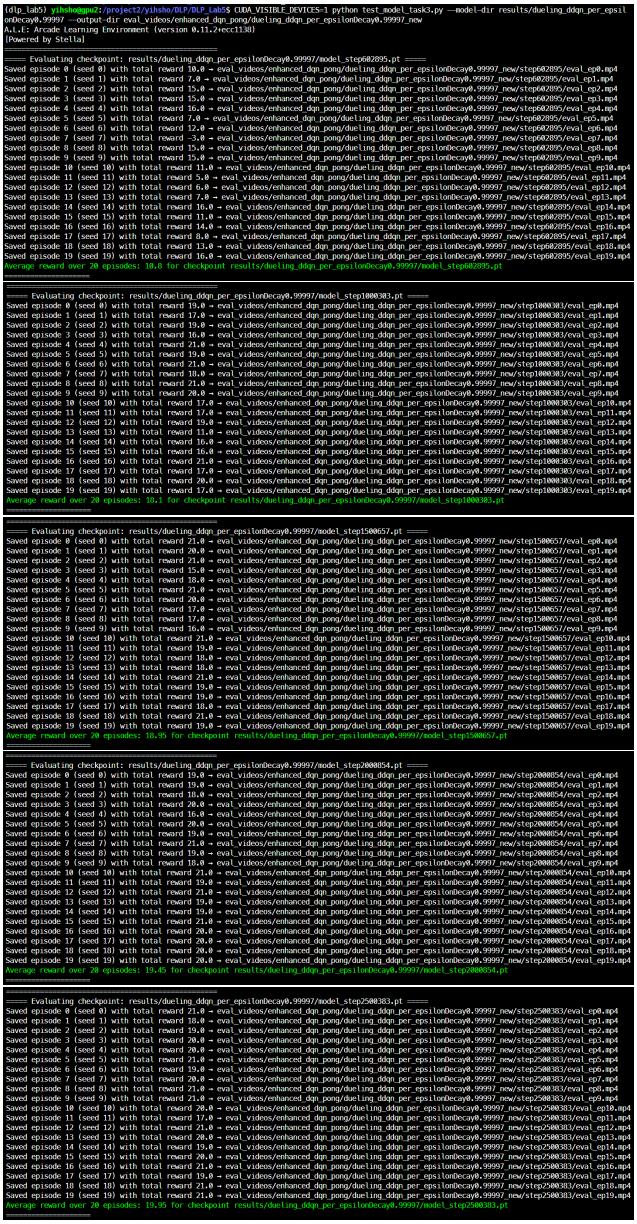


Figure 11. **Evaluation for Enhanced Pong (Task 3).** Checkpoints were saved at 602895, 1000303, 1500657, 2000854, 2500383 environment steps.

3.2. Sample Efficiency Analysis

The DQN enhancements dramatically improved sample efficiency. As shown in Fig. 12, the baseline agent in Task 2 required approximately 800k steps to reach a positive evaluation score. In contrast, the enhanced agent in Task 3 achieved a positive score around 350k to 400k steps, effectively doubling the learning speed. Furthermore, the enhanced agent reached a stable winning plateau (19 or higher) at 1.5M steps, while the baseline agent took nearly 4M steps to reach a stable plateau.

This significant improvement is primarily due to:

1. **Multi-step returns**: Propagating rewards backward over 3 steps instead of 1, allowing for faster credit assignment.
2. **Prioritized Experience Replay (PER)**: Ensuring that the agent revisits transitions with high TD-errors, focusing the learning on the most informative samples.
3. **Dueling Architecture**: Better separating the state-value estimation from action-specific advantages, which is particularly useful in Pong where many states have similar values regardless of the move.

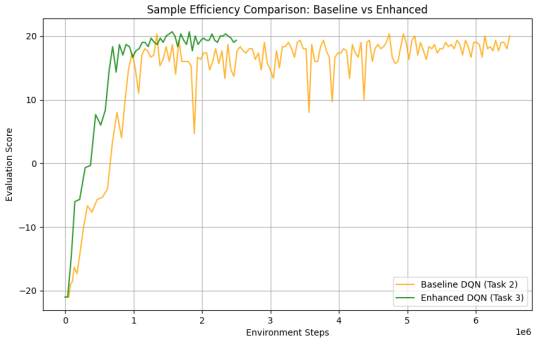


Figure 12. Sample efficiency comparison between Baseline DQN (Task 2) and Enhanced DQN (Task 3).

3.3. Ablation Study

Method	600k	1M	1.5M	2M	2.5M
Baseline DQN	0.2	16.45	16.1	16.8	16.45
Enhanced (All)	5.25	14.95	18.2	**19.4**	18.85
Enhanced 2 (All)	10.33	16.67	17.33	**19**	18
w/o DDQN	1.67	7	5.67	15	17.33
w/o Dueling	18.33	16.67	18.33	17	**19**
w/o PER	4.33	7.67	12	16.33	13
w/o Multi-step	9.67	14.33	18.33	17.33	18.67
w/o Act-Reduce	6.33	16.67	16.33	17.33	18
w/o Act-Reduce 2	10.8	18.1	18.95	**19.45**	**19.95**

Table 2. **Comparison of different methods at specific training steps**. The score above 19 is marked in bold.

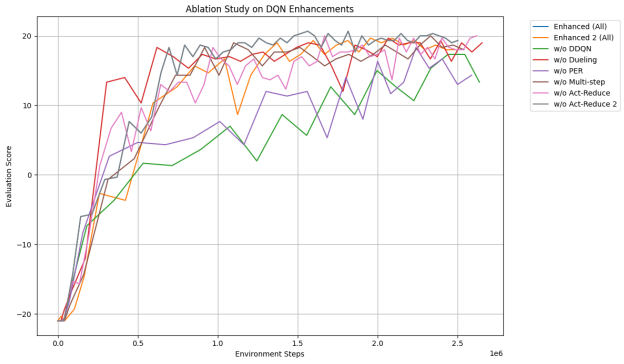


Figure 13. Ablation study of DQN enhancements on Pong.

I performed an ablation study by removing each technique individually from the Task 3 configuration. The results in Fig. 13 and Tab. 2 show:

- **w/o PER**: Most severe degradation, peaking at only 13.0. This confirms PER's critical role in prioritizing informative samples in sparse reward environments.
- **w/o Double DQN**: Significantly slowed early learning and limited the final score to 17.33, highlighting its importance in mitigating overestimation bias.
- **w/o Dueling**: Fastest early-stage learner (18.33 at 600k steps) but reached a similar final plateau (19.0) as the enhanced versions.
- **w/o Multi-step**: Showed a slower start but eventually reached 18.67, indicating its primary benefit is accelerating early-stage credit assignment.
- **High Variance**: Configurations like *w/o Act-Reduce 2* reached the highest final score (19.95), suggesting that training outcome is highly sensitive to initialization and exploration noise.

4. Conclusion

In this lab, I implemented and evaluated a series of DQN enhancements to solve the Pong-v5 environment. The combination of Double DQN, Dueling DQN, PER, and Multi-step returns significantly outperformed the baseline, achieving a 2x improvement in sample efficiency and much higher final scores.

Through the ablation study, I identified Prioritized Experience Replay (PER) and Double DQN as the most critical enhancements for reaching high performance and stability. The experiment also revealed substantial variance in training outcomes, where factor like random initialization sometimes allowed configurations without action space reduction to achieve the highest scores (+19.95). Overall, this lab demonstrated that while sophisticated algorithmic enhancements provide a robust foundation, careful hyperpa-

rameter tuning and an understanding of stochastic factors remain vital in deep reinforcement learning.

DISCLOSURE FOR AI TOOLS

I used Copilot to help me write the code and report, and use Claude for ideas to improve learning efficiency. However, I have checked and modified them to ensure the correctness and quality.